Notatki do powtórki

Projekt „CNN od zera w numpy”: wszystko, co sprawiło trudność w krokach 1–6 planu

Ta notatka zbiera miejsca z naszej pracy nad projektem, w których odpowiedź na pytanie kontrolne była błędna lub niepełna, w których napisałeś, że czegoś nie rozumiesz, albo w których błąd w kodzie wymagał wyjaśnienia. Każdy temat ma najpierw wersję prostą, a potem szczegóły ze wzorami, kształtami tablic i przykładem liczbowym.

ŹRÓDŁO I ZAKRES

Dla tego projektu istnieje jeden transkrypt sesji i to z niego pochodzą wszystkie cytaty. Obejmuje on kroki 1–6 planu (dane, warstwy gęste, gradient check, SGD z baseline’em, konwolucja w wersji naiwnej). Na pytania kontrolne z kroków 4 i 5 nie odpowiadałeś, a kroków 7–17 jeszcze nie zaczęliśmy, więc nie ma ich w notatce. Tematy, które zrozumiałeś od razu, pominąłem.

Spis treści

A. PYTANIA KONTROLNE

1. Poziomice kosztu bez normalizacji
2. Dev i test z tego samego rozkładu
3. Skąd bierze się ŷ – y
4. Kształt dW i podwójne 1/m
5. Różnica dwustronna, błąd względny, koszt gradientu numerycznego
6. Backward konwolucji: suma, akumulacja, stride

B. DANE

7. Średnia i odchylenie: skalar czy wektor
8. Podział na train, dev i test
9. Mini-batche, generatory i yield
10. Po co tasować co epokę

C. PYTHON

11. Klasy, obiekty i self
12. Notebook: stare moduły i nadpisane zmienne

D. WARSTWY I BACKPROP

13. Skąd się bierze dZ i co zwraca backward
14. Iloczyn element po elemencie, norma, maximum
15. Softmax z cross-entropy
16. Flatten

E. GRADIENT CHECK

17. Jak działa: perturbacja jednej współrzędnej
18. Jak czytać wynik i szukać przyczyny

F. TRENING I EWALUACJA

19. Jeden krok SGD
20. Ewaluacja i dokładność

G. KONWOLUCJA

21. Forward: wymiary, stride, padding, kanały
22. Backward: dW, db i dA
23. Dlaczego wersja naiwna jest tak wolna

A. PYTANIA KONTROLNE

1. Poziomice kosztu bez normalizacji

CO NAPISAŁEŚ

Krok 1, pytanie 2. Odpowiedź: *„mają wiele lokalnych minimów, w których gradient descent mógłby utknąć”*.

CO BYŁO NIE TAK

Brak normalizacji nie tworzy lokalnych minimów. Zmienia kształt jednej doliny: robi ją długą i wąską.

Prosto

Wyobraź sobie rynnę: w poprzek jest bardzo stroma, a wzdłuż prawie płaska. Kulka puszczona z boku odbija się od ścianki do ścianki i prawie nie posuwa się wzdłuż rynny. Tak zachowuje się gradient descent, gdy cechy mają różne skale. Normalizacja zamienia rynnę w miskę, w której kulka toczy się prosto do środka.

Szczegóły

Poziomica to linia łącząca punkty o tym samym koszcie, tak jak warstwica na mapie. Gdy jedna cecha ma wartości dużo większe niż druga, koszt reaguje na jej wagę dużo mocniej i poziomice stają się wydłużonymi elipsami.

```
Przykład:  J(w₁, w₂) = w₁² + 100·w₂²
Gradient:  (2·w₁, 200·w₂)

Krok w kierunku w₂ jest stabilny tylko dla α < 2/200 = 0,01.
Przy α = 0,01 waga w₁ maleje w każdym kroku tylko o 2%:
w₁ ← (1 − 0,02)·w₁, więc zmniejszenie jej 100 razy wymaga około 228 kroków.
```

Jedno α musi obsłużyć oba kierunki naraz. Stromy kierunek wymusza małe α, a przez to w płaskim kierunku idziemy bardzo wolno. Gradient wskazuje głównie w poprzek doliny, stąd zygzak. Po normalizacji oba kierunki mają podobną krzywiznę i jedno α pasuje do obu.

JAK SPRAWDZIĆ

Wytrenuj baseline dwa razy z tym samym α: raz na danych znormalizowanych, raz na surowych wartościach 0–255. Na surowych koszt powinien spadać wyraźnie wolniej albo się rozbiegać.

2. Dev i test z tego samego rozkładu

CO NAPISAŁEŚ

Krok 1, pytanie 3. Odpowiedź: *„ponieważ na dev chcemy dobrać hiperparametry tak, żeby model jak najlepiej sprawdzał się na test”*.

CO BYŁO NIE TAK

To opis celu, a nie powodu. Brakowało odpowiedzi na pytanie, co się stanie, gdy rozkłady będą różne.

Prosto

Dev to tarcza, do której celujesz podczas strojenia. Test to tarcza, na której zostaniesz oceniony. Jeśli stoją w różnych miejscach, nauczysz się trafiać nie tam, gdzie trzeba.

Szczegóły

- „Rozkład” oznacza tu rodzaj danych: skąd pochodzą, jak wyglądają, jak często występuje każda klasa.
- Każdy wybór hiperparametru to wybór modelu najlepszego na dev. Jeśli dev różni się od test, wybierzesz model najlepszy dla niewłaściwych danych, a wynik na test będzie niespodzianką.
- Test nie służy do strojenia w ogóle. Używasz go raz, na końcu. Każde spojrzenie na test i poprawka po nim sprawia, że przestaje być niezależną oceną.
- W projekcie dev pochodzi z pliku treningowego (5 000 z 60 000 przykładów), a test z osobnego pliku, ale oba są z tego samego zbioru Fashion-MNIST, więc warunek jest spełniony.

JAK SPRAWDZIĆ

Porównaj histogramy klas dla dev i test (funkcja `plot_distribution`). Na końcu projektu wynik na test powinien być bliski wynikowi na dev. Wyraźnie gorszy oznacza przeuczenie do dev.

3. Skąd bierze się ŷ – y

CO NAPISAŁEŚ

Krok 2, pytanie 1. Odpowiedź: *„z reguły łańcuchowej dL/dŷ · dŷ/da […] wszystko się upraszcza do takiej postaci”*.

CO BYŁO NIE TAK

Kierunek dobry, ale bez pokazania, co i dlaczego się upraszcza.

Prosto

Koszt karze za małe prawdopodobieństwo poprawnej klasy. Softmax zamienia liczby z, zwane logitami, na prawdopodobieństwa ŷ. Gdy połączysz oba kroki, pochodna wychodzi bardzo prosta: „to, co model przewidział, minus to, co powinien”.

Szczegóły

$$
\begin{aligned}
&\text{Softmax:} && \hat{y}_k = \exp(z_k) \,/\, \Sigma_j \exp(z_j) \\
&\text{Koszt:} && L = -\Sigma_k\, y_k \cdot \log \hat{y}_k \qquad \text{(y to one-hot: jedna jedynka, reszta zera)} \\
&\text{Pochodna softmaxu:} && \partial\hat{y}_k/\partial z_j = \hat{y}_k\cdot(\delta_{kj} - \hat{y}_j) \qquad (\delta_{kj} = 1 \text{ gdy } k = j, \text{ inaczej } 0) \\
&\text{Pochodna kosztu:} && \partial L/\partial \hat{y}_k = -\, y_k/\hat{y}_k
\end{aligned}
$$

$$
\begin{aligned}
\partial L/\partial z_j &= \Sigma_k\, (-\, y_k/\hat{y}_k) \cdot \hat{y}_k\cdot(\delta_{kj} - \hat{y}_j) \\
&= -\,\Sigma_k\, y_k\cdot\delta_{kj} + \hat{y}_j \cdot \Sigma_k\, y_k \\
&= -\, y_j + \hat{y}_j \cdot 1 \qquad \text{(bo } \Sigma_k\, y_k = 1\text{)} \\
&= \hat{y}_j - y_j
\end{aligned}
$$

Kluczowe są dwa skrócenia: $\hat{y}_k$ z mianownika kosztu znosi się z $\hat{y}_k$ z pochodnej softmaxu, a suma jedynek w wektorze one-hot daje 1. Dla całego batcha o m przykładach koszt jest średnią, więc `dZ = (Ŷ − Y)/m`, o kształcie (m, liczba klas).

```
Przykład:  ŷ = [0,7  0,2  0,1],  y = [1  0  0]   →   ŷ − y = [−0,3  0,2  0,1]
Logit poprawnej klasy trzeba zwiększyć, pozostałe zmniejszyć.
```

JAK SPRAWDZIĆ

Gradient check dla ostatniej warstwy. Dodatkowo każdy wiersz dZ powinien sumować się do zera.

4. Kształt dW i podwójne 1/m

CO NAPISAŁEŚ

Krok 2, pytania 2 i 3. Najpierw dwa razy *„nie wiem”*. Po wskazówkach: *„n_in, n_out, zgadza się”* oraz *„gradient dostaje dwa razy 1/m i rozmiar batcha ma wtedy wpływ na trening”*.

CO BYŁO NIE TAK

Druga próba była poprawna. Warto utrwalić uzasadnienie, bo za pierwszym razem go zabrakło.

Prosto

Gradient mówi, jak zmienić każdą wagę z osobna, więc musi mieć tyle samo pól co macierz wag. A dzielenie przez liczbę przykładów m wolno zrobić tylko raz, bo robimy z kosztu średnią jeden raz.

Szczegóły

```
Forward:   Z = A·W + b        A: (m, n_in)   W: (n_in, n_out)   Z: (m, n_out)
Backward:  dW = Aᵀ·dZ         Aᵀ: (n_in, m)  dZ: (m, n_out)  →  dW: (n_in, n_out)

Przykład:  A (4, 5), dZ (4, 2)  →  Aᵀ·dZ ma kształt (5, 4)·(4, 2) = (5, 2), czyli jak W.
```

Waga W[i, j] łączy wejście numer i z wyjściem numer j. Jej gradient to suma po wszystkich przykładach iloczynów „wartość wejścia i" razy „gradient wyjścia j". Mnożenie $A^T \cdot dZ$ liczy dokładnie te sumy dla wszystkich par (i, j) naraz. Jeśli kształty się nie zgadzają, to znak, że mnożysz w złej kolejności albo bez transpozycji.

Podwójne 1/m. W projekcie 1/m jest tylko w gradiencie kosztu (`dZ = (Ŷ − Y)/m`). Warstwy sumują po batchu i nie dzielą. Gdyby warstwa dzieliła jeszcze raz, gradient miałby $1/m^2$. Dla batcha 32 byłby 32 razy za mały, a ten sam α dawałby inny krok przy każdym rozmiarze batcha. Gradient check wykryłby to od razu: różnica wyszłaby dokładnie o czynnik m.

JAK SPRAWDZIĆ

Asercje kształtów w `backward` (masz je w `Dense`). Gradient check na batchu większym niż 1, bo dla m = 1 błąd z 1/m jest niewidoczny.

5. Różnica dwustronna, błąd względny, koszt gradientu numerycznego

CO NAPISAŁEŚ

Krok 3. (1) *„ponieważ uśredniamy pochodną z przesunięcia do przodu i do tyłu"*. (2) *„błąd względny może być różny dla różnych wartości parametru i ciężko stwierdzić, czy się mieści w normie"*. (3) *„żeby policzyć gradient numeryczny, musielibyśmy najpierw zrobić krok do przodu"*.

CO BYŁO NIE TAK

(1) Opis bez powodu. (2) Niejasne, brak sedna. (3) Błędny powód: forward i tak wykonujemy, problemem jest ich liczba.

Prosto

- Patrząc w obie strony od punktu, błędy „w prawo" i „w lewo" w dużej mierze się znoszą.
- Błąd względny odpowiada na pytanie „jaka część wartości jest zła", więc jedna granica pasuje do dużych i małych gradientów.
- Gradient numeryczny wymaga dwóch przejść przez sieć dla każdej wagi osobno. Backprop daje wszystkie wagi za jednym razem.

Szczegóły

```
Rozwinięcie Taylora:
J(θ + ε) = J + ε·J′ + (ε²/2)·J″ + (ε³/6)·J‴ + …
J(θ − ε) = J − ε·J′ + (ε²/2)·J″ − (ε³/6)·J‴ + …

Jednostronnie:  (J(θ+ε) − J(θ)) / ε      = J′ + (ε/2)·J″ + …      błąd rzędu ε
Dwustronnie:    (J(θ+ε) − J(θ−ε)) / (2ε) = J′ + (ε²/6)·J‴ + …     błąd rzędu ε²

Przykład: J(x) = x³ w punkcie x = 1 (prawdziwa pochodna 3), ε = 0,1
jednostronnie:  (1,331 − 1) / 0,1       = 3,31   (błąd 0,31)
dwustronnie:    (1,331 − 0,729) / 0,2   = 3,01   (błąd 0,01)
```

Przy odejmowaniu znikają wyrazy z parzystą potęgą ε, w tym największy składnik błędu (ε²/2)·J″.

```
błąd = ‖g − g_num‖ / (‖g‖ + ‖g_num‖)        ‖v‖ = √(Σ vᵢ²)   (norma, patrz temat 14)

Dlaczego nie bezwzględny:
gradient około 1000,  różnica 0,0001   →  względnie 5·10⁻⁸   (świetnie)
gradient około 10⁻⁶,  różnica 10⁻⁷     →  względnie 0,05     (źle)
Błąd bezwzględny oceniłby te dwa przypadki odwrotnie.
```

Koszt. Twój baseline ma 784·128 + 128 + 128·10 + 10 = 101 770 parametrów. Gradient numeryczny to 2 przejścia forward na parametr, czyli 203 540 przejść na jeden krok uczenia. Backprop to jeden forward i jeden backward. Do tego różnice skończone mają szum zaokrągleń, więc nadają się tylko do sprawdzania.

JAK SPRAWDZIĆ

Uruchom `grad_check` z ε równym 10^{-5}, 10^{-6} i 10^{-7} i zobacz, jak zmienia się błąd (patrz temat 18).

6. Backward konwolucji: suma, akumulacja, stride

CO NAPISAŁEŚ

Krok 6. Pytania 1 i 2: *„nie wiem, wytłumacz mi to prostym językiem”*. Pytanie 3: *„taki sam jak to, czego jest gradientem, nie wiem dokładnie dlaczego”*.

CO BYŁO NIE TAK

Dwa pytania bez odpowiedzi, trzecie poprawne, ale bez uzasadnienia.

Prosto

Filtr to pieczątka odbijana w wielu miejscach obrazka. Każde odbicie mówi coś o tym, jak pieczątkę poprawić, więc zbieramy uwagi ze wszystkich miejsc (suma w dW). Z kolei jeden piksel obrazka trafia pod pieczątkę kilka razy, bo odbicia na siebie nachodzą. Każde odbicie oddaje mu swój kawałek gradientu, więc dokładamy je do siebie (akumulacja w dA).

Szczegóły

```
Przykład 1D: wejście x₁ x₂ x₃ x₄, filtr (w₁, w₂), stride 1
z₁ = w₁·x₁ + w₂·x₂
z₂ = w₁·x₂ + w₂·x₃
z₃ = w₁·x₃ + w₂·x₄

dw₁ = dz₁·x₁ + dz₂·x₂ + dz₃·x₃        (w₁ użyta trzy razy → trzy składniki)
dx₂ = dz₁·w₂ + dz₂·w₁                 (x₂ leży w dwóch oknach → dwa składniki)

Ten sam przykład ze stride 2:
z₁ = w₁·x₁ + w₂·x₂,   z₂ = w₁·x₃ + w₂·x₄
dx₂ = dz₁·w₂                          (x₂ leży już tylko w jednym oknie)
```

- **Stride** to odstęp między oknami. Większy stride oznacza mniej okien, więc każdy piksel dostaje mniej składników gradientu. Piksel, który nie trafił do żadnego okna, dostaje zero.
- **Kształty.** Gradient ma zawsze kształt tego, po czym liczymy pochodną, bo dla każdego elementu mówi, jak jego zmiana wpływa na koszt: dW jak W, czyli (f, f, n_C_prev, n_C); db jak b, czyli (n_C,); dA_prev jak wejście, czyli (m, n_H_prev, n_W_prev, n_C_prev).

Pełne wzory 2D są w temacie 22.

JAK SPRAWDZIĆ

Gradient check dla konfiguracji (stride, padding): (1, 0), (1, 1) i (2, 1). Do tego asercje kształtów trzech gradientów.

B. DANE

7. Średnia i odchylenie: skalar czy wektor

CO SIĘ WYDARZYŁO

Pytałeś: *„odchylenie standardowe i średnia to mają być skalary?”*, *„co znaczy żeby przejść na skalary przy std”*, *„po normalizacji średnia powinna być 0 i std 1 po jakiej osi”*. W kodzie pojawiły się kolejno: statystyki liczone przed podzieleniem przez 255, asercja `np.sum(std) == 0`, średnia po pikselach z odchyleniem dzielonym przez m, a potem dzielnik `m · X.size`.

CO BYŁO NIE TAK

Mieszały się dwa warianty normalizacji, a dzielnik we wzorze na odchylenie nie odpowiadał liczbie sumowanych wartości.

Prosto

Skalar to jedna liczba. W projekcie liczysz jedną średnią i jedno odchylenie ze wszystkich pikseli wszystkich obrazków treningowych i tymi samymi dwiema liczbami przekształcasz każdy piksel. Odchylenie mówi, jak bardzo wartości są rozrzucone wokół średniej, więc dla prawdziwych danych jest zawsze większe od zera.

Szczegóły

```
N = liczba wszystkich wartości = m · 784 = X.size

μ = (1/N) · Σ x                σ = √( (1/N) · Σ (x − μ)² )
x̃ = (x/255 − μ) / σ            μ i σ liczone po przeskalowaniu do [0, 1]
```

Wariant	Po czym liczysz	Co wychodzi	Uwagi
Skalar (wybrany)	po wszystkich elementach	jedna μ i jedna σ	brak dzielenia przez zero
Wektor po pikselach	po osi przykładów (axis=0)	784 wartości μ i σ	piksel zawsze czarny ma σ = 0, dzielenie daje nan lub inf

- **Dzielnik musi być liczbą sumowanych wartości.** Dzielnik `m · X.size` to $m^2 \cdot 784$, więc σ wyszło $\sqrt{m}$ razy za małe. Po normalizacji odchylenie zbioru train wyniosło 234,5 zamiast 1, a $\sqrt{55\,000} \approx 234{,}5$.
- **Asercja „suma std równa 0" była odwrócona.** Poprawny warunek to σ > 0 i wartość skończona.
- **Kolejność:** najpierw dzielenie przez 255, potem μ i σ. Inaczej odejmujesz średnią ze skali 0–255 od wartości ze skali 0–1.
- **Po jakiej osi sprawdzać:** dla wariantu skalarnego po wszystkich elementach naraz. Średnia ≈ 0 i odchylenie ≈ 1 wychodzą dokładnie tylko na train. Na dev i test są bliskie, ale nie identyczne.

JAK SPRAWDZIĆ

`check_norm` na `X_train`: średnia około 0, odchylenie 1,0. μ ma być pojedynczą liczbą. Brak nan w danych.

8. Podział na train, dev i test

CO SIĘ WYDARZYŁO

W kolejnych wersjach `load_data`: dev był wycinany ze zbioru testowego, wycinek miał indeks zmiennoprzecinkowy (`m_test * 0.5`), dev był liczony jako `X_train[:-dev_size]`, a etykiety dev miały zgubiony dwukropek w wycinku.

CO BYŁO NIE TAK

Dev nakładał się na train (50 000 wspólnych przykładów zamiast 5 000 osobnych) albo zmniejszał zbiór testowy.

Prosto

Trzy zbiory mają trzy różne zadania i nie mogą mieć wspólnych przykładów. Na train model się uczy. Na dev wybierasz ustawienia. Test otwierasz raz, na końcu, jak zapieczętowaną kopertę. Jeśli dev zawiera przykłady z train, model „zna odpowiedzi" i wynik na dev jest zawyżony.

Szczegóły

```
Po przetasowaniu 60 000 przykładów z pliku treningowego jedną permutacją:
train = pozycje od 0 do 54 999         (55 000 przykładów)
dev   = pozycje od 55 000 do 59 999    (5 000 przykładów)
test  = cały osobny plik               (10 000 przykładów, nietknięty)
```

- X i Y muszą być tasowane tą samą permutacją, inaczej obrazki dostaną cudze etykiety.

- Indeks wycinka musi być liczbą całkowitą. Wynik mnożenia przez 0,5 jest zmiennoprzecinkowy i numpy go odrzuca.
- `tablica[n]` zwraca jeden element, a `tablica[n:]` wszystkie od pozycji n. Zgubiony dwukropek dał jedną etykietę zamiast 5 000.
- μ i σ liczysz po podziale i tylko na train. Inaczej informacja z dev i test wycieka do danych treningowych.

JAK SPRAWDZIĆ
Kształty: (55000, 28, 28, 1), (5000, 28, 28, 1), (10000, 28, 28, 1) oraz etykiety (m, 10). Długości X i Y w każdym zbiorze muszą być równe.

9. Mini-batche, generatory i yield

CO SIĘ WYDARZYŁO
Pytania: *„czym się różni yield od return”*, *„jak przerwać pętlę, jeśli nie ma kolejnego next”*. Błędy: `next()` wywołane na krotce generatorów, `yield` po pętli zamiast w niej, zwracanie `next(get_batch(...))` (tylko pierwsza paczka), suma 54 990 zamiast 55 000, a później *„uruchomiłem kod i nic się nie stało”* po wywołaniu `fit`.

CO BYŁO NIE TAK
Generator był traktowany jak zwykła funkcja albo jak gotowa lista.

Prosto

Funkcja z `return` jest jak automat, który wydaje jedną rzecz i się wyłącza. Funkcja z `yield` jest jak podajnik: za każdym razem, gdy poprosisz, wydaje następną rzecz i czeka. Gdy rzeczy się skończą, podajnik jest pusty i nie da się go napełnić. Trzeba wziąć nowy.

Szczegóły

- **Wywołanie funkcji z yield niczego nie liczy.** Tworzy tylko generator. Kod rusza dopiero przy pobieraniu wartości, czyli w pętli `for` albo przez `next`. Dlatego samo wywołanie `fit(...)` nic nie wypisało.
- **yield musi być w pętli.** Jeden `yield` po pętli oddaje jedną wartość (u Ciebie całą listę paczek) i generator się kończy.
- **Generator jest jednorazowy.** Nowa epoka to nowy generator. Każde `next` zużywa jedną paczkę na stałe.
- **Koniec danych** to wyjątek `StopIteration`. Pętla `for` obsługuje go sama. Przy ręcznym `next` trzeba go złapać albo podać wartość domyślną.
- **Ostatnia paczka może być krótsza.** Liczba paczek to m/rozmiar zaokrąglone w górę.

```
Przykład: m = 10, rozmiar 4  →  3 paczki: 4, 4 i 2 przykłady.

Zagadka 54 990: m = 55 000, rozmiar 10  →  5 500 pełnych paczek.
Brakowało dokładnie 10 przykładów, czyli jednej paczki.
Zużyło ją wcześniejsze next(train) w innej komórce notebooka.
```

JAK SPRAWDZIĆ
`check_minibatch_len` na świeżo utworzonym generatorze: suma długości paczek równa m, żadna paczka nie jest pusta, X i Y w paczce mają tę samą długość.

10. Po co tasować co epokę

CO NAPISAŁEŚ

„po co mam tasować jeszcze raz?”, a po pierwszym wyjaśnieniu: *„dalej nie rozumiem, po co mam jeszcze raz tasować”*.

CO BYŁO NIE TAK

Pierwsze wyjaśnienie było zbyt ogólne. Pomógł dopiero konkretny przykład.

Prosto

Jeśli zawsze uczysz się fiszek w tych samych zestawach i w tej samej kolejności, najlepiej pamiętasz to, co było na końcu, i uczysz się kolejności zamiast treści. Przetasowanie talii przed każdą powtórką usuwa ten efekt.

Szczegóły

```
8 przykładów, batch 4, bez tasowania w każdej epoce:
paczka 1: A B C D        paczka 2: E F G H

Każdy krok przesuwa wagi w stronę dobrą dla bieżącej paczki.
Epoka zawsze kończy się na paczce 2, więc model jest zawsze lekko
przechylony w stronę E–H. To błąd systematyczny: w każdej epoce ten sam.

Z tasowaniem:  epoka 1: {A E B G} {C H D F},  epoka 2: {D A H C} {F B E G}, …
Przechylenia mają różne kierunki i między epokami się znoszą.
```

- Gradient z mini-batcha to przybliżenie gradientu z całego zbioru. Przybliżenie jest uczciwe tylko wtedy, gdy skład paczek jest losowy.
- Jedno tasowanie w `load_data` przygotowuje tylko pierwszą epokę. Dlatego `get_minibatch` dostaje ten sam obiekt `rng` w każdej epoce. Gdyby dostawał liczbę (seed), każda epoka miałaby identyczną kolejność.
- Dev i test nie wymagają tasowania, bo tam niczego się nie uczysz.

JAK SPRAWDZIĆ

Dwa kolejne generatory z tym samym `rng` powinny dać różne pierwsze paczki. Eksperyment: ten sam model z tasowaniem co epokę i bez, porównanie krzywych kosztu.

C. PYTHON

11. Klasy, obiekty i self

CO NAPISAŁEŚ

„nie rozumiem, jak ma działać ta klasa, generalnie prawie nic nie wiem o klasach”. Błędy po drodze: wywołanie `MyClass.__init__(a=1, b=3)` i `MyClass.add(c=3)` na klasie, wejście A w konstruktorze `Dense`, warunek `if __name__` wewnątrz klasy, `model.parameters` bez nawiasów, `SoftmaxCrossEntropy` przekazane bez nawiasów, komunikat *„takes 3 positional arguments but 4 were given”*.

CO BYŁO NIE TAK

Mylenie przepisu (klasy) z konkretną rzeczą zrobioną według przepisu (obiektem).

Prosto

Klasa to przepis na ciasto. Obiekt to upieczone ciasto. Z jednego przepisu możesz upiec dwa ciasta i zjedzenie jednego nie zmienia drugiego. Przepisu nie da się zjeść, więc metody wywołujesz na cieście, czyli na obiekcie. Słowo `self` wewnątrz klasy oznacza „to konkretne ciasto, na którym właśnie pracuję".

Szczegóły

- **Tworzenie obiektu:** nazwa klasy z nawiasami. Python sam uruchamia `__init__` i sam podaje nowy obiekt jako `self`. Ręczne wołanie `__init__` na klasie nie tworzy obiektu.
- **Nawiasy mają znaczenie.** `SoftmaxCrossEntropy` to przepis, `SoftmaxCrossEntropy()` to obiekt. `model.parameters` to sama metoda, `model.parameters()` to jej wynik. Stąd błąd o brakującym argumencie Y: metoda wołana na klasie nie dostała `self`.
- **Stan w self.** To, co zapiszesz jako `self.coś`, zostaje w obiekcie między wywołaniami. Dlatego warstwa zapisuje wejście w `self.cache` podczas `forward` i korzysta z niego w `backward`.
- **Co do konstruktora, co do metody.** Konstruktor dostaje to, co stałe (wagi, stride, padding). Metoda `forward` dostaje to, co zmienia się przy każdym wywołaniu (batch danych).
- **„3 argumenty, a podano 4".** Przy wywołaniu `obiekt.metoda(a, b, c)` Python dokłada obiekt jako pierwszy argument. Metoda zdefiniowana bez `self` dostaje więc o jeden argument za dużo.
- **Jedna metoda woła drugą** przez `self.nazwa(...)`. Sama nazwa szuka funkcji poza klasą.
- **Blok `if __name__ == "__main__"`** stoi na końcu pliku, poza klasą. Kod w nim uruchamia się tylko przy bezpośrednim uruchomieniu pliku, a nie przy imporcie.

JAK SPRAWDZIĆ

Utwórz dwa obiekty tej samej klasy z różnymi wartościami i wywołaj na nich tę samą metodę. Wyniki powinny być różne, a zmiana jednego obiektu nie powinna wpływać na drugi.

12. Notebook: stare moduły i nadpisane zmienne

CO SIĘ WYDARZYŁO

„dlaczego nie mogę zaimportować Data z pliku data.py", *„dlaczego jak usunąłem z run_conv_naive relu, to […] dalej pokazuje to relu"*, błąd kształtów (5000, 10) i (10000, 10) przy ewaluacji, plik `main.py` uruchamiający trening przy imporcie.

CO BYŁO NIE TAK

Notebook pamiętał starą wersję kodu albo stare obiekty, a jedna zmienna została przypisana dwa razy.

Prosto

Notebook ma pamięć. Raz zaimportowany plik siedzi w niej w starej wersji, dopóki nie zrestartujesz kernela. To samo dotyczy zmiennych: wytrenowany wcześniej model nie zmieni się od tego, że poprawisz plik.

Szczegóły

- **Import jest zapamiętywany.** Po zmianie pliku `.py` zrestartuj kernel albo włącz automatyczne przeładowanie (`autoreload`) na początku notebooka.

- **Obiekt to nie kod.** Zmienna `model` wskazuje na obiekt zbudowany starą funkcją. Żeby zobaczyć zmianę, zbuduj model od nowa.
- **Dwa przypisania do jednej nazwy.** W komórce było `X_preview1, X_preview2, X_preview2, Y_preview1, …`. Druga wartość nadpisała pierwszą, więc `X_preview2` zawierało zbiór testowy (10 000), a `Y_preview2` etykiety dev (5 000). Stąd niezgodność (5000, 10) i (10000, 10).
- **Kod na poziomie pliku uruchamia się przy imporcie.** Dlatego trening trafił do funkcji (`run_MLP`), a nie leży luzem w `main.py`.

JAK SPRAWDZIĆ

Wypisz nazwy klas warstw w `model.layers` i kształty wszystkich sześciu tablic zaraz po `load_data`. Przy dziwnym błędzie najpierw restart kernela i uruchomienie komórek od góry.

D. WARSTWY I BACKPROP

13. Skąd się bierze dZ i co zwraca backward

CO SIĘ WYDARZYŁO

Pytania: *„okej, a skąd wezmę dz”*, *„Z powinienem zapisać w self, czy jako argument do forward i backward”*. Błędy: `Dense.backward` zwracał dZ zamiast dA, `backward` bez `return`, klucze słownika bez cudzysłowów (`self.cache[A]`, `self.grads[dW]`), `Sigmoid.forward` bez `return`, backward konwolucji zwracający trzy wartości.

CO BYŁO NIE TAK

Niejasne, co przepływa między warstwami w przód, a co w tył.

Prosto

Sieć to łańcuch ludzi podających sobie paczki. W przód każdy dostaje paczkę z danymi, przerabia ją i podaje dalej, zapamiętując, co dostał. W tył każdy dostaje od następnego karteczkę „jak twoje wyjście wpływa na koszt”, przelicza ją na „jak twoje wejście wpływa na koszt” i podaje poprzedniemu.

Szczegóły

```
Forward:   X → Dense₁ → Z₁ → ReLU → A₁ → Dense₂ → Z₂ → koszt J
Backward:  koszt daje dZ₂ = (Ŷ − Y)/m
           Dense₂.backward(dZ₂)  zwraca dA₁    (i zapisuje dW₂, db₂)
           ReLU.backward(dA₁)    zwraca dZ₁ = dA₁ ⊙ 1[Z₁ > 0]
           Dense₁.backward(dZ₁)  zwraca dX     (i zapisuje dW₁, db₁)

Dense:   dW = Aᵀ·dZ      db = suma dZ po przykładach      dA = dZ·Wᵀ
         (n_in, n_out)   (n_out,)                         (m, n_in)
```

- **Warstwa nie wytwarza dZ sama.** Dostaje gradient z warstwy wyżej, a pierwszy gradient w łańcuchu daje funkcja kosztu.
- **backward zwraca jedną rzecz:** gradient względem swojego wejścia. Gradienty parametrów zapisuje w `self.grads`. Zwrócenie dZ zamiast dA cofa łańcuch o krok za daleko, a brak `return` przerywa go (następna warstwa dostaje `None`).
- **Cache:** to, czego `backward` potrzebuje z `forward`, zapisujesz w obiekcie, bo w tył przepływają już tylko gradienty. Dense potrzebuje A, ReLU potrzebuje Z, tanh i sigmoid wygodniej zapamiętują wynik A.

- **Klucze słownika to napisy** w cudzysłowie. Bez cudzysłowu Python szuka zmiennej o tej nazwie, a tablica numpy nie może być kluczem.

```
Pochodne aktywacji (wszystko element po elemencie):
ReLU:     dZ = dA ⊙ 1[Z > 0]
tanh:     dZ = dA ⊙ (1 − A²)          A = tanh(Z)
sigmoid:  dZ = dA ⊙ A ⊙ (1 − A)       A = 1/(1 + e^(−Z))   (uwaga na minus w wykładniku)
```

JAK SPRAWDZIĆ

Kształt zwracanej wartości ma być równy kształtowi wejścia warstwy. Sigmoid dla Z = 0 daje 0,5, a dla Z = 2 około 0,88. Gradient check z każdą aktywacją osobno.

14. Iloczyn element po elemencie, norma, maximum

CO NAPISAŁEŚ

O wzorze dZ = dA·(1 − A²): *„to ma być iloczyn skalarny?”*. O wzorze na błąd: *„te dwie pionowe kreski to abs?”*. W kodzie ReLU: `np.max(0, Z)`.

CO BYŁO NIE TAK

Trzy podobnie wyglądające zapisy oznaczają różne działania.

Prosto

Mnożenie „pole po polu” daje tablicę tej samej wielkości. Iloczyn skalarny mnoży pola parami i wszystko sumuje do jednej liczby. Podwójne kreski to długość wektora, a pojedyncze to wartość bezwzględna jednej liczby.

Szczegóły

```
a = [1, 2, 3],  b = [4, 5, 6]

element po elemencie (⊙, w numpy gwiazdka):    [4, 10, 18]      ten sam kształt
iloczyn skalarny:                              4 + 10 + 18 = 32  jedna liczba
iloczyn macierzowy (np.dot lub @):             wiersze razy kolumny, zmienia kształt

norma:      ‖[3, 4]‖ = √(3² + 4²) = 5          (np.linalg.norm)
wart. bezwzględna:   |−3| = 3
```

- W pochodnych aktywacji i w maskach zawsze chodzi o mnożenie element po elemencie: każdy neuron ma własną pochodną.
- Iloczyn macierzowy pojawia się w warstwie gęstej (A·W, A^T·dZ, dZ·W^T).
- `np.max(tablica, oś)` szuka największej wartości w jednej tablicy, a drugi argument to numer osi. `np.maximum(0, Z)` porównuje dwie rzeczy element po elemencie i to jest ReLU.

JAK SPRAWDZIĆ

Sprawdzaj kształt wyniku: po mnożeniu element po elemencie musi być taki jak kształt składników.

15. Softmax z cross-entropy

CO SIĘ WYDARZYŁO

Pierwsze wersje: suma po całej tablicy zamiast po wierszach, maksimum bez `keepdims`, koszt `log(Ŷ·Y + eps)`. Pytania: o zdanie, że backward zakłada połączenie z cross-entropy (*„co to znaczy”*), *„czyli w forward mam od razu liczyć koszt?”*, *„ale przecież ja potrzebuję y_hat, a nie log y_hat”*. Gradient check dał błędy rzędu 10^{-4}.

CO BYŁO NIE TAK

Kilka pułapek numerycznych i jedna koncepcyjna: ta warstwa to dwa kroki sklejone w jeden.

Prosto

Softmax zamienia dowolne liczby na prawdopodobieństwa, które w każdym wierszu sumują się do 1. Cross-entropy patrzy tylko na prawdopodobieństwo poprawnej klasy i karze tym mocniej, im jest ono mniejsze. W projekcie oba kroki są w jednej klasie, bo wtedy pochodna jest prosta (temat 3) i obliczenia są stabilne.

Szczegóły

```
Dla każdego wiersza (przykładu) osobno, Z ma kształt (m, liczba klas):

1. przesunięcie:   Z′ = Z − max(Z w wierszu)          wynik softmaxu się nie zmienia
2. log-softmax:    log Ŷ = Z′ − log Σ exp(Z′)         suma po klasach, z keepdims
3. softmax:        Ŷ = exp(log Ŷ)
4. koszt:          J = −(1/m) · Σ Y ⊙ log Ŷ           suma po wszystkich polach
5. backward:       dZ = (Ŷ − Y)/m                     bez argumentów, Ŷ i Y są w cache
```

- **Po wierszach, nie po całości.** Suma i maksimum idą po osi klas (axis=1). `keepdims=True` zostawia kształt (m, 1), dzięki czemu odejmowanie i dzielenie dopasowują się do wierszy, a nie do kolumn.
- **Po co przesunięcie.** exp(1000) przekracza zakres liczb i daje inf. Po odjęciu maksimum największy wykładnik to 0, więc nic się nie przepełnia.
- **Dlaczego nie eps w logarytmie.** Gradient (Ŷ – Y)/m odpowiada kosztowi bez eps. Gdy poprawna klasa ma małe prawdopodobieństwo, eps wyraźnie zmienia koszt i przestaje on pasować do gradientu.

```
Twój przypadek: ŷ poprawnej klasy = 6,4·10⁻⁶, eps = 10⁻⁸
względna zmiana:  10⁻⁸ / 6,4·10⁻⁶ ≈ 1,6·10⁻³
Gradient check pokazał błędy około 6·10⁻⁴. Po usunięciu eps spadły do 4·10⁻⁹.
```

- **Potrzebujesz obu:** log Ŷ do kosztu i Ŷ do gradientu. Liczysz najpierw logarytm z logitów, a Ŷ dostajesz przez exp.
- **„Zakłada połączenie”:** `backward` tej klasy nie jest pochodną samego softmaxu (ta byłaby macierzą dla każdego przykładu). Jest pochodną softmaxu razem z kosztem. Dlatego klasa musi być ostatnia w sieci i nie należy do listy warstw w `Sequential`.
- **Koszt liczysz w forward,** bo tylko tam spotykają się Ŷ i etykiety Y. Potrzebujesz go do monitorowania i do gradient checku.

JAK SPRAWDZIĆ

Wiersze Ŷ sumują się do 1. Dla losowych małych wag J ≈ ln 10 ≈ 2,30. Logity ±1000 dają skończony koszt. Gradient check poniżej 10^{-7}.

16. Flatten

CO SIĘ WYDARZYŁO

Pytanie: *„na jakim etapie używa się flatten”*. W kodzie `forward` używał `A.flatten()`.

CO BYŁO NIE TAK

`A.flatten()` spłaszcza wszystko, razem z osią przykładów.

Prosto

Warstwa gęsta przyjmuje dla każdego przykładu jeden rząd liczb, a obrazek jest prostokątem z kanałami. Flatten rozwija każdy obrazek w jeden długi rząd, ale nie miesza obrazków ze sobą.

Szczegóły

```
Forward:   (m, 28, 28, 1)  →  (m, 784)          oś przykładów zostaje
Backward:  (m, 784)        →  (m, 28, 28, 1)    powrót do zapamiętanego kształtu

Błąd:      A.flatten() dla (m, 28, 28, 1) daje jeden wektor (m·784,)
```

- Flatten stoi na przejściu z części „obrazkowej” do gęstej: jako pierwsza warstwa w baseline’ie i po konwolucji w CNN.
- Nie ma parametrów. W cache wystarczy zapamiętać kształt wejścia.

JAK SPRAWDZIĆ

Wejście (2, 3, 4, 1) powinno dać (2, 12), a `backward` z powrotem (2, 3, 4, 1).

E. GRADIENT CHECK

17. Jak działa: perturbacja jednej współrzędnej

CO NAPISAŁEŚ

„co znaczy perturbować”, *„nie rozumiem błędu 1”*, *„czyli będziemy sprawdzać tylko część wartości z każdego parametru?”*, *„to jak dobrać n_samples, skoro każdy parametr może mieć inny shape?”*, *„ale jak utworzyliśmy params jako kopię, to w pętli nie zmieniamy parametrów modelu”*. W kodzie: zaburzanie X zamiast wag, zaburzanie wszystkich wylosowanych wag naraz, `orig − 2·eps`, brak przywrócenia wagi.

CO BYŁO NIE TAK

Niejasne, co dokładnie zmieniamy, ile rzeczy naraz i na którym obiekcie.

Prosto

Perturbować znaczy lekko trącić. Żeby sprawdzić, jak jedno pokrętło wpływa na dźwięk, kręcisz tylko nim, odrobinę w prawo i odrobinę w lewo, słuchasz różnicy i odkręcasz z powrotem. Jeśli ruszysz trzema pokrętłami naraz, nie wiesz, które co zmieniło.

Szczegóły

```
Dla jednej wagi θ:   g_num = (J(θ + ε) − J(θ − ε)) / (2ε)
Kolejność:  zapamiętaj θ  →  ustaw θ + ε, policz J⁺  →  ustaw θ − ε, policz J⁻
            →  przywróć θ  →  zapisz g_num

Dlaczego po jednej: J = w₁² + w₂² + w₃², wagi 1, 2, 3
prawdziwe pochodne:              2, 4, 6
wszystkie trzy trącone naraz:    2 + 4 + 6 = 12, wpisane każdej z trzech wag
```

- **Zaburzasz wagi, nie dane.** Sprawdzasz dW i db, więc zmieniasz W i b. Zmienianie X testuje co innego (gradient względem wejścia).
- **Referencja, nie kopia.** `model.parameters()` zwraca te same tablice, których używają warstwy. Zmiana w miejscu od razu wpływa na następny `forward`. Gdybyś zaburzał kopię, J^+ i J^- byłyby równe, a gradient numeryczny zerowy.
- **Dolna perturbacja to θ – ε** liczone od oryginału, nie θ – 2ε.
- **n_samples to górny limit na parametr:** sprawdzasz min(n_samples, rozmiar) losowych pól. Małe parametry (biasy) sprawdzają się w całości, duże są próbkowane. Każde pole kosztuje dwa pełne przejścia forward.
- **Biasy mają jeden wymiar,** wagi dwa lub cztery. Dlatego w pętli trzymasz całą krotkę współrzędnych, bez rozbijania jej na wiersz i kolumnę.
- **Dzielenie przez zero:** gdy oba gradienty są zerowe, błąd przyjmujesz jako 0.

Zauważone przy pisaniu notatki, wcześniej nieomawiane: w `grad_check` warunek `if n_samples == −1: n_samples = param.size` działa tylko dla pierwszego parametru. Po nim `n_samples` ma już wartość 72 (rozmiar filtrów konwolucji) i przy `n_samples = −1` wagi `Dense` są sprawdzane na 72 polach z 62 720, a nie w całości. Zgadza się to z czasem około 33 s, który widziałeś.

JAK SPRAWDZIĆ

Po zakończeniu checku wagi mają być identyczne jak przed nim. Mała sieć i float64: wszystkie błędy poniżej 10^{-7}.

18. Jak czytać wynik i szukać przyczyny

CO SIĘ WYDARZYŁO

Trzy sytuacje: błędy około $6 \cdot 10^{-4}$ (eps w logarytmie), próba sabotażu przez usunięcie transpozycji, oraz stały błąd około $5{,}7 \cdot 10^{-7}$ dla konwolucji, o który pytałeś *„takie błędy są ok?”*.

CO BYŁO NIE TAK

Sam wynik nie mówi, gdzie leży przyczyna. Pomaga sprawdzenie, jak błąd zależy od ε.

Prosto

Gradient check to waga kontrolna w sklepie. Jeśli wskazania się różnią, zepsuty może być towar (twój `backward`), sama waga (koszt lub precyzja liczb) albo sposób ważenia. Zmiana ε działa jak ważenie tego samego towaru na kilka sposobów: wzór, według którego zmienia się różnica, zdradza winnego.

Szczegóły

Błąd względny	Znaczenie
poniżej 10^{-7}	backward poprawny
10^{-7} do 10^{-5}	szara strefa, trzeba zbadać
powyżej 10^{-3}	prawie na pewno błąd

Gdy ε rośnie, błąd…	Najbardziej prawdopodobna przyczyna
maleje	szum zaokrągleń w J (dzielony przez większe ε), kod jest dobry
rośnie	błąd przybliżenia pochodnej (rzędu ε^2), kod jest dobry
stoi w miejscu	stała różnica po stronie analitycznej: błąd w kodzie albo za mała precyzja liczb

Twoje pomiary dla sieci z konwolucją:

ε	conv W	conv b	Dense W	Dense b
10^{-7}	$5{,}7\cdot10^{-7}$	$5{,}5\cdot10^{-7}$	$5{,}7\cdot10^{-9}$	$3{,}3\cdot10^{-9}$
10^{-6}	$5{,}7\cdot10^{-7}$	$5{,}5\cdot10^{-7}$	$5{,}4\cdot10^{-10}$	$3{,}0\cdot10^{-10}$
10^{-5}	$5{,}7\cdot10^{-7}$	$5{,}5\cdot10^{-7}$	$7{,}6\cdot10^{-11}$	$4{,}5\cdot10^{-11}$

- **Dense:** błąd maleje ze wzrostem ε, czyli zwykłe zaokrąglenia. **Konwolucja:** błąd stoi. Przyczyną były tablice gradientów tworzone jako `float32` w `backward` konwolucji. float32 pamięta około 7 cyfr znaczących, więc sumy w pętli gubiły dokładność na poziomie 10^{-7}. Po zmianie na float64 błąd spadł do około $1{,}2\cdot10^{-8}$.
- **Dlaczego nie „dziedziczyć typu”.** `zeros_like(W)` kopiuje typ W. Gdy W jest tablicą liczb całkowitych (tak było w Twoim teście), gradient też jest całkowity i dodawanie ułamków kończy się błędem. Typ gradientów ustaw jawnie na float64.
- **Dane też w float64.** `load_data` zwraca float32, więc przed checkiem trzeba je rzutować.
- **Sprawdzenie samego testu.** Usunięcie transpozycji w dW wykrył błąd kształtów, zanim check cokolwiek porównał, więc część numeryczna nie została sprawdzona. Błąd, który nie psuje kształtów (dA = dZ·W bez transpozycji przy kwadratowej macierzy), wstawiony w drugiej warstwie `Dense` dał błędy 0,62 i 0,83 dla parametrów pierwszej warstwy. Ten sam błąd w pierwszej warstwie był niewidoczny, bo jej dA nikomu nie jest przekazywane.
- **Etykiety wyników.** Rozpoznawanie „W” po liczbie wymiarów równej 2 oznaczało filtry konwolucji (4 wymiary) jako „b”.

JAK SPRAWDZIĆ

Przy błędzie w szarej strefie uruchom check dla trzech wartości ε. Wypisz typy (`dtype`) wag, cache i gradientów.

19. Jeden krok SGD

CO NAPISAŁEŚ

„jak uzgodnić wymiary params z grads”, *„a wag nie powinienem aktualizować po przejściu przez minibatch?”*, *„najpierw mam zrobić forward czy backward?”*, *„połączyłem optimizer z fit, nie wiem czy słusznie”*. W kodzie: lista parametrów pobrana raz przed pętlą, `np.dot(d_param, X)` przy aktualizacji wag, wersja bez aktualizacji, pętla po krotce dwóch list.

CO BYŁO NIE TAK

Kolejność kroków i moment, w którym gradienty są aktualne.

Prosto

Jeden krok uczenia to zawsze ta sama sekwencja na jednej paczce danych: policz wynik, zmierz błąd, ustal, kto ile zawinił, popraw wagi. Potem bierzesz następną paczkę. Poprawka następuje po każdej paczce, nie raz na epokę.

Szczegóły

```
Dla każdego mini-batcha (X, Y):
1. Z  = model.forward(X)            wypełnia cache w warstwach
2. J  = loss.forward(Z, Y)          koszt, zapamiętuje Ŷ i Y
3. dZ = loss.backward()             (Ŷ − Y)/m
4. model.backward(dZ)               wypełnia gradienty w warstwach
5. θ ← θ − α·gradient               dla każdej pary z model.parameters()
```

- **Forward zawsze przed backward,** bo backward czyta z cache to, co zapisał forward.
- **Nic nie trzeba „uzgadniać”.** `model.parameters()` zwraca gotowe pary (parametr, jego gradient) o tych samych kształtach.
- **Gradient jest już gotowy.** dW = $A^T \cdot dZ$ policzył `backward`. Ponowne mnożenie przez X przy aktualizacji to błąd. Wzór jest ten sam dla wag i biasów.
- **Pobieraj pary po każdym backward.** `backward` tworzy za każdym razem nowe tablice gradientów. Lista pobrana przed pętlą wskazuje na gradienty z pierwszej paczki.
- **Aktualizacja w miejscu.** Zapis ze znakiem `-=` zmienia tablicę, na którą patrzy warstwa. Zapis „parametr = parametr − …” tworzy nową tablicę, a warstwa dalej używa starej.
- **Po epoce zamiast po paczce** byłoby innym, wolniejszym algorytmem: jedna poprawka na 55 000 przykładów zamiast około 860.
- **Podział ról:** `step_SGD` tylko poprawia wagi, a `fit` prowadzi pętlę. Momentum i Adam z kroku 11 będą potrzebowały własnej pamięci, więc wtedy optymalizator stanie się obiektem.

JAK SPRAWDZIĆ

Po jednym kroku wagi mają się różnić od poprzednich, a koszt na tej samej paczce spaść (u Ciebie z 2,17 do 1,90). Test przeuczenia na 32 przykładach: koszt bliski zera i 100% dokładności.

20. Ewaluacja i dokładność

CO NAPISAŁEŚ

„co ma robić funkcja evaluate”, *„wystarczy, że policzę acc na jednym minibatchu?”*, *„martwi mnie, że jak robię forward, żeby policzyć accuracy, to wpływa to później na gradienty”*. W kodzie: `np.sum(np.where(Y_hat == Y))`, porównanie numerów klas z tablicą one-hot, `argmax` z wartości kosztu.

CO BYŁO NIE TAK

Sposób liczenia dokładności i obawa o wpływ pomiaru na uczenie.

Prosto

Ewaluacja to sprawdzian bez poprawiania: model odpowiada, ty liczysz trafienia, niczego w nim nie zmieniasz. Dokładność to ułamek trafionych odpowiedzi. Jedna paczka 32 przykładów to za mało pytań, żeby ocena była wiarygodna.

Szczegóły

```
przewidziana klasa = numer największego logitu w wierszu Z
prawdziwa klasa    = numer jedynki w wierszu Y (one-hot)
dokładność         = średnia z porównania tych dwóch wektorów numerów klas

Przykład: przewidziane [0, 1], prawdziwe [0, 2]  →  trafienia [tak, nie]  →  0,5
```

- **Softmax nie jest potrzebny do predykcji,** bo nie zmienia kolejności wartości: największy logit to największe prawdopodobieństwo.
- **Porównuj numery klas z numerami klas.** Porównanie numerów z tablicą one-hot daje bezsensowny wynik. `np.where` zwraca pozycje trafień, a ich suma to nie liczba trafień.
- **Koszt to jedna liczba,** nie tablica przewidywań. `argmax` bierze się z Z.

```
Niepewność pomiaru (błąd standardowy) przy dokładności 0,9:
32 przykłady:     √(0,9·0,1/32)   ≈ 0,053   czyli około ±5 punktów procentowych
5 000 przykładów: √(0,9·0,1/5000) ≈ 0,004   czyli około ±0,4 punktu procentowego
```

- **Czy forward psuje gradienty?** Sam forward nie zmienia wag ani gradientów. Zmienia tylko cache. Szkodzi wyłącznie wtedy, gdy wykonasz go między forward i backward tej samej paczki treningowej, bo backward policzy wtedy gradienty z cudzych danych. Ewaluacja po zakończeniu pętli po paczkach jest bezpieczna.
- **Średnia po przykładach, nie po paczkach.** Ostatnia paczka jest krótsza, więc zwykła średnia z wyników paczek lekko przekłamuje. Lepiej zliczać trafienia i dzielić przez liczbę przykładów. To wciąż jest do poprawy w `fit`.

Wynik baseline'u: 93% na train i 90% na dev (55 000 i 5 000 przykładów, 50 epok, batch 64, $\alpha = 0{,}01$). Luka około 3 punktów oznacza brak silnego przeuczenia.

JAK SPRAWDZIĆ

`accuracy` dla przykładu powyżej ma zwrócić 0,5. Model z losowymi wagami powinien mieć około 10% (10 klas).

21. Forward: wymiary, stride, padding, kanały

CO SIĘ WYDARZYŁO

Pytania: *„czyli mam napisać tę funkcję tylko dla padding same?”*, *„po co się tak rozszerza te obrazy, zwiększając liczbę kanałów”*, *„co jak licznik n_H wychodzi nieparzysty”*, *„jak wybrać wymiary, które chcę spadować w np.pad”*, *„a co jeśli licznik nie jest podzielny przez 2”*. W kodzie: okno bez mnożenia przez stride, wynik wpisywany do wszystkich kanałów naraz, brak biasu, padding jako liczba zmiennoprzecinkowa.

CO BYŁO NIE TAK

Arytmetyka pozycji okna i rola poszczególnych osi tablicy.

Prosto

Filtr to małe okienko z wagami, które przesuwasz po obrazku. W każdym położeniu mnożysz to, co widać w okienku, przez wagi, sumujesz i dostajesz jedną liczbę. Każdy filtr tworzy własną „mapę”, która pokazuje, gdzie na obrazku występuje jedna cecha, na przykład pionowa krawędź. Liczba kanałów to liczba takich map, a nie rozmiar obrazka.

Szczegóły

```
Kształty:  A_prev (m, n_H_prev, n_W_prev, n_C_prev)      W (f, f, n_C_prev, n_C)
           b (n_C,)                                      Z (m, n_H, n_W, n_C)

Rozmiar wyjścia:   n_H = (n_H_prev + 2p − f)/s + 1     (licznik musi dzielić się przez s)

Jedna wartość wyjścia:
Z[i, h, w, c] = Σ ( okno ⊙ W[:, :, :, c] ) + b[c]
okno = A_pad[i,  h·s … h·s + f − 1,  w·s … w·s + f − 1,  wszystkie kanały]
```

- **Początek okna to h·s, nie h.** Bez mnożenia przez stride kod działa tylko dla s = 1.
- **Każdy filtr ma własny kanał wyjścia i własny bias.** Wpisanie wyniku do wszystkich kanałów naraz zostawia w nich wynik ostatniego filtra.
- **Padding to parametr,** dowolna liczba całkowita. „Same” to tylko wybór takiego p, żeby rozmiar się nie zmienił: dla s = 1 jest to p = (f − 1)/2, całkowite dla nieparzystego f. Używaj dzielenia całkowitego, bo `np.pad` nie przyjmuje ułamków.
- **Nieparzysty łączny padding** (możliwy przy s > 1) można rozdzielić nierówno: mniej z góry i z lewej, więcej z dołu i z prawej. Backward musi wtedy użyć tego samego podziału.
- **Parzystość n_H nie ma znaczenia.** Liczy się podzielność licznika przez stride. Gdy jej brak, skrajne piksele nie trafiają do żadnego okna.
- **np.pad dostaje parę (przed, po) dla każdej osi.** Dla obrazków zera dokładasz tylko do wysokości i szerokości: (0, 0), (p, p), (p, p), (0, 0). Oś przykładów i oś kanałów zostają bez zmian.
- **Po co więcej kanałów:** więcej filtrów to więcej wykrywanych cech. W głębszych warstwach mapy maleją, a kanałów przybywa: informacja „gdzie” jest ściskana, informacja „co” rośnie. Filtr głębszej warstwy obejmuje wszystkie kanały wejścia, więc łączy proste cechy w złożone.
- **W numpy nie ma operatora konwolucji.** W sieciach „konwolucja” to w praktyce korelacja krzyżowa: filtr nie jest odwracany.

```
Przykłady, które sprawdziliśmy (filtr 3×3 z samych jedynek, bias 0, p = 0):
wejście 4×4 z liczbami 0…15, s = 1  →  [[45, 54], [81, 90]]
wejście 5×5 z liczbami 0…24, s = 2  →  [[54, 72], [144, 162]]
wejście 4×4, s = 2                  →  licznik 4 − 3 = 1 nie dzieli się przez 2, asercja
słusznie przerywa
```

JAK SPRAWDZIĆ

Powyższe przykłady policzone ręcznie. Dwa filtry 1×1 z biasami 10 i 20 na pikselu o wartości 0 powinny dać [10, 20]. Padding „same” przy s = 1 zachowuje rozmiar.

22. Backward: dW, db i dA

CO NAPISAŁEŚ

„wytłumacz mi backward”, *„dobrze sobie opisałem te wymiary?”*, *„a forward w cache powinienem zapisać a prev po czy przed paddingiem”*, *„dobrze napisałem dW?”*, *„co jest nie tak z dA_pad”*.

CO BYŁO NIE TAK

W komentarzu dZ miało n_C_prev zamiast n_C. Pętle szły po rozmiarach wejścia, a nie wyjścia. Pojedynczy piksel był mnożony przez wycinek dZ (odwrotnie niż trzeba). Pętla po filtrach miała zakres kanałów wejścia. Wejście z cache nie było dopadowane. dA_pad miało kształt bez ramki, stąd błąd kształtów (3, 2, 1) i (3, 3, 1).

Prosto

Idziesz po tych samych położeniach okna co w forward. W każdym położeniu masz jedną liczbę z dZ, która mówi, jak ważne było to miejsce. Okno wejścia pomnożone przez tę liczbę dokładasz do gradientu filtra. Filtr pomnożony przez tę liczbę dokładasz do gradientu wejścia, w miejscu, gdzie leżało okno.

Szczegóły

```
dZ ma kształt wyjścia: (m, n_H, n_W, n_C)        (n_C = liczba filtrów)

Dla każdego przykładu i, położenia (h, w) i filtra c:
dW[:, :, :, c]                    +=  okno(i, h, w) · dZ[i, h, w, c]      okno: (f, f,
n_C_prev)
db[c]                             +=  dZ[i, h, w, c]
dA_pad[i, miejsce okna (h, w), :] +=  W[:, :, :, c] · dZ[i, h, w, c]

Na końcu: dA_prev = dA_pad bez ramki o grubości p   (dla p = 0 bez wycinania)
```

- **Pętle po wyjściu.** Położeń okna jest tyle, ile pól ma wyjście (n_H na n_W), a filtrów tyle, ile kanałów ma dZ.
- **Okno razy liczba, nie piksel razy wycinek.** W forward okno i filtr dały jedną liczbę. W backward ta jedna liczba (z dZ) wraca do całego okna i do całego filtra.
- **„+=”, nie „=”.** Okna nachodzą na siebie, więc to samo pole wejścia dostaje kilka składników (temat 6).
- **Praca w przestrzeni z ramką.** Położenia okien liczone są na obrazku z paddingiem. W cache zapisujesz wejście bez ramki i p, a w backward dopadowujesz je ponownie. dA_pad musi mieć kształt wejścia z ramką. Tablica bez ramki jest za mała i skrajne okno zostaje obcięte: stąd wycinek (3, 2, 1) zamiast (3, 3, 1).

- **Dla p = 0** wycinek „od p do –p” jest pusty, więc ten przypadek obsługujesz osobno.
- **Interfejs jak w Dense:** `backward` zwraca tylko dA, a dW i db zapisuje w `self.grads`. Stride i padding ustawiasz w konstruktorze, bo `Sequential` woła `forward` z jednym argumentem.
- **Typ gradientów float64** (temat 18).

JAK SPRAWDZIĆ

Kształty: dW jak W, db jak b, dA jak wejście. Gradient check na małym obrazku (na przykład 6×6, 2 filtry, 2–3 przykłady) dla kilku par stride i padding. U Ciebie po poprawkach: około $1{,}2 \cdot 10^{-8}$.

23. Dlaczego wersja naiwna jest tak wolna

CO NAPISAŁEŚ

„dałem jedną epoch i strasznie długo to trwa i nic się nie printuje”, *„czekam już ponad 2 min i dalej nic”*, *„jak dobrać n_samples do grad check dla tego modelu”*, *„dałem 10 przykładów, ile to będzie trwać”*.

CO BYŁO NIE TAK

Wersja z pętlami została użyta do treningu na całym zbiorze, a wypis pojawia się dopiero po zakończeniu epoki.

Prosto

Wersja naiwna liczy każde okienko osobno, w zwykłej pętli Pythona. To jak przepisywanie książki litera po literze zamiast kserowania całych stron. Nadaje się do zrozumienia i do sprawdzania gradientów na malutkich danych, ale nie do treningu.

Szczegóły

```
Zmierzony czas forward: około 10 ms na jeden obrazek 28×28 (8 filtrów 3×3)
Na obrazek przypada 28·28·8 = 6 272 okien liczonych w pętli Pythona.

forward na 55 000 przykładach train:  około 9 minut
forward na 5 000 przykładach dev:     około 1 minuty

Jedna epoka = trening (forward i backward na każdej paczce)
            + ewaluacja całego train + ewaluacja całego dev
            = kilkadziesiąt minut, a wypis pojawia się dopiero na końcu epoki.
```

```
Gradient check tego modelu: dwa przejścia forward na każde sprawdzane pole.
10 przykładów → około 0,1 s na forward → około 0,2 s na pole.

Rozmiary parametrów: filtry 3·3·1·8 = 72, bias 8, Dense 6272·10 = 62 720, bias 10.
limit 50 pól na parametr:  50 + 8 + 50 + 10 = 118 pól     →  około 25 sekund
wszystkie pola:            62 810 pól                     →  około 3,5 godziny
```

- Pojedyncze wywołanie numpy na dużej tablicy wykonuje pętlę w szybkim, skompilowanym kodzie. Pętla Pythona płaci stały narzut przy każdym obrocie.
- Do testów pętli treningowej z naiwną konwolucją wystarczy kilkaset przykładów i jedna lub dwie epoki.
- Pełny gradient check rób na małej sieci testowej. Na dużym modelu ustaw limit kilkudziesięciu pól na parametr.

- Rozwiązaniem docelowym jest wektoryzacja z kroku 8 planu, po której wersja naiwna zostaje jako wzorzec do porównań.

JAK SPRAWDZIĆ

Zmierz czas forward na 10 przykładach i przelicz na cały zbiór, zanim uruchomisz trening. Po wektoryzacji porównaj wyniki obu wersji na tych samych danych: powinny być równe z dokładnością do zaokrągleń.

CZEGO NIE DAŁO SIĘ USTALIĆ

- Pytania kontrolne z kroku 4 (mini-batch SGD) i kroku 5 (forward konwolucji): w transkrypcie nie ma Twoich odpowiedzi.
- Kroki 7–17 (pooling, wektoryzacja, optymalizatory, batch norm, regularyzacja, ResNet): jeszcze nierozpoczęte.
- Test przeuczenia na 32 przykładach z kroku 4: w transkrypcie nie ma wyniku.
- Dokładne wartości kosztu i dokładności baseline'u w kolejnych epokach: znane są tylko wartości końcowe, w przybliżeniu 93% i 90%.
- Wcześniejsze sesje projektu: w katalogu transkryptów jest tylko jedna sesja, więc jeśli istniały inne rozmowy, nie są tu uwzględnione.